Looped Transformers and Recurrent Depth: Eight Years from Universal Transformers to GPT-6 Astra

Five core papers · a map of sixty related works · ten trend insights

awesome_loop_transformer · https://github.com/asimfish/awesome_loop_transformer

2026-09-06

Contents

Abstract 1

How to read this report 1

1. Why now 1

2. Concepts and taxonomy 2
- 2.1 What a looped transformer is . . . 2
- 2.2 Four loop granularities . . . 2
- 2.3 Three orthogonal axes . . . 2

3. An eight-year timeline 3

4. The five core papers at a glance 3
- 4.1 How the five fit together . . . 4

5. The wider literature map 4
- 5.1 Theory: expressivity, ICL, length generalization . . . 4
- 5.2 Scaled pretraining . . . 4
- 5.3 Efficiency: latency, KV memory, quantization . . . 5
- 5.4 Stability and training theory . . . 5
- 5.5 Adaptive computation and halting . . . 5
- 5.6 Retrofit and training-free . . . 5
- 5.7 Post-training and reinforcement learning . . . 5
- 5.8 Interpretability and dynamics . . . 6
- 5.9 Tiny recursive reasoners . . . 6
- 5.10 Cross-modal and applications . . . 6

6. Ten trend insights 6
- Insight 1: Recurrent depth passed the compute-matching gate — through a narrow door . . . 6
- Insight 2: “How many loops” splits into two regimes . . . 7
- Insight 3: Loop granularity is getting finer and converging on the middle . . . 7
- Insight 4: ODE flow or fixed-point iteration? Two philosophies are diverging . . . 7
- Insight 5: Training stability went from folklore to theorems . . . 8
- Insight 6: Inference cost is the last big bottleneck, now decomposed into three sub-problems . . . 8
- Insight 7: Latent looping and explicit CoT are complements, not substitutes . . . 8
- Insight 8: The capability profile is now clear — loops manipulate knowledge, they do not store it . . . 8
- Insight 9: Applications are expanding from math puzzles to agents and embodiment . . . 8
- Insight 10: Productization has started; the toolchain has not caught up . . . 8

7. Inferences about GPT-6 Astra (explicitly labeled as inference) 9

8. Open problems 9

9. Conclusion 9

Appendix A: Circulation errors corrected in this report 9

Appendix B: Method and sources 10

Appendix C: Glossary 10

Abstract

Reusing one set of parameters along the depth axis — an idea introduced by the Universal Transformer in 2018 — became one of the fastest-growing directions in large-model architecture research in 2025–2026, and reached public attention in September 2026 when The Information reported that GPT-6 Astra uses "recurrent depth". This report takes the five papers threaded together by the WeChat article "The Loop technology behind GPT-6 Astra" as its skeleton, re-reads each against the original text to restore the preconditions and numbers that secondary coverage dropped, and then uses an exhaustive arXiv search to place roughly 60 directly related works from 2018–2026 into ten themes and ten trend judgments. Three conclusions stand out. First, recurrent depth has for the first time beaten a vanilla MoE transformer under **measured compute matching** (Loopie, July 2026), but the win sits in a narrow configuration: two steps, per-layer looping, MoE backbone. Second, "how many loops" splits into two regimes — under a fixed pretraining budget the answer is 2; with variable inference compute the answer is "more, adaptively" — and LoopCoder-v2's "three or more regress" is the price of a parallel-loop design, not a universal ceiling on recurrent depth. Third, latent looping and explicit chain-of-thought are complementary axes of test-time compute, and stacking them yields more than the sum of their parts.

How to read this report

Three reading paths, by available time:

Time	Read	You get
5 minutes	Abstract → the headings and bold sentences of §6 → Appendix A	Where the field stands and which popular claims are inaccurate
30 minutes	§2–4 (concepts, timeline, five-paper table) → all of §6 → §8 open problems	A complete map; you can place a new paper in its cell
2 hours	Everything + the five reports in `reports/en/01–05`	Every key number traceable to a table in the original paper

Unfamiliar terms are defined in **Appendix C**. Diagram labels are in Chinese; each caption carries the full meaning. Everything about GPT-6 Astra is labeled as inference.

1. Why now

On 2 September 2026 The Information reported that OpenAI's newly released GPT-6 Astra uses an architectural technique called recurrent depth. Researcher @nrehiew_ then traced the lineage on X through five papers, and the WeChat account PaperAgent turned that thread into a Chinese long-form article. This report's direct goal is to read those five papers closely and put them back into the larger literature.

On Astra itself this report does exactly one thing: separate what can be verified from what cannot. **Verifiable**: the contents, numbers, and relations of the five papers. **Not verifiable**: which kind of looping Astra uses, how many loops, how it was trained — OpenAI's release page discloses no architectural detail, The Information's report rests on anonymous sources, and neither primary document was accessible to us. Every statement about Astra below is labeled as inference.

2. Concepts and taxonomy

2.1 What a looped transformer is

In a standard transformer forward pass each layer's parameters are used once; depth equals parameter count divided by per-layer parameters. A **looped transformer** (also looped / recurrent-depth / weight-tied / universal transformer) applies the same parameters repeatedly, decoupling depth from parameter count: a 4-layer block looped 32 times has an effective depth of 128 layers.

2.2 Four loop granularities

Granularity	Definition	Representatives
Model-loop	The whole stack's output is fed back as input	Universal Transformer, Huginn's core, Ouro
Block-loop	Only a contiguous middle segment loops; first and last layers do not	Huginn's prelude–core–coda, Hyperloop, Training-Free Looped, SMELT
Layer-loop	Each layer repeats K times before passing to the next	Loopie, Training-Free layer-mode, MixerLoop (mixer only)
Latent conditioning	The previous loop's (or previous token's) hidden state is injected as a condition	Coconut, PLT's CLP, Latent Recurrent Transformer



Figure 1 (labels in Chinese): model-loop repeats the whole stack; block-loop repeats only the middle segment with input injection; layer-loop repeats each layer before the next. Effective depth = stored layers × loops, parameters unchanged.

These are not exclusive. Loopie is layer-loop; Huginn is block-loop at the three-stage level and model-loop inside its core; PLT is model-loop plus latent conditioning (the previous round's state shifted by one position).

2.3 Three orthogonal axes

Beyond granularity, three independent design axes:

1. **Fixed versus adaptive loop count.** Fixed (every token gets the same compute: Loopie, LoopCoder-v2); random at training, free at inference (Huginn); per-token adaptive halting (UT's ACT, Mixture-of-Recursions, Think-at-Hard, RecurTrace).
2. **From scratch, retrofit, or training-free.** Pretrain from scratch (Huginn, Ouro, Loopie, Nanbeige4.2); convert an existing model and continue training (Relaxed Recursive Transformers, Retrofitted Recurrence); no training at all, wrap at inference (Training-Free Looped Transformers, Program-of-Layers).
3. **What the objective treats the loop as.** Plain language modeling with truncated unrolling (Huginn); fixed point / equilibrium (DEQ, Attractor Models, Fixed-Point Reasoners, STARS); refinement of an ODE integration (Training-Free); latent CoT with intermediate supervision (LOTUS, LoopMTP); RL with rewards on loop steps (RLTT, LoopRPT).

3. An eight-year timeline

When	Event
2016	Neural GPU (Kaiser & Sutskever), Adaptive Computation Time (Graves): precursors of looping + adaptive halting
2018-07	**Universal Transformers** (Dehghani et al.): parameters shared across time steps, coordinate embeddings, ACT; ICLR 2019
2019	ALBERT uses cross-layer sharing for compression; Deep Equilibrium Models replace explicit unrolling with fixed-point solving
2021–2022	Deep-thinking line (Schwarzschild, Bansal): random unrolling + input injection lets recurrent nets extrapolate easy→hard; path-independent equilibrium models (Anil)
2023	Looped Transformers as Programmable Computers (Giannou): a 13-layer looped transformer emulates a general computer; Looped Transformers are Better at Learning Learning Algorithms (Yang): <10% parameters match standard ICL; Sparse Universal Transformer
2024	MoEUT: first shared-layer model to slightly beat standard transformers on LM; Relaxed Recursive Transformers (per-layer LoRA); Looped Transformers for Length Generalization; Coconut reasons in continuous space
2025-02	**Scaling up Test-Time Compute with Latent Reasoning** (Geiping et al.): Huginn-3.5B, 800B tokens, coins "recurrent depth"; Reasoning with Latent Thoughts (Saunshi) proves k layers looped L times ≈ kL layers
2025-05	To CoT or To Loop (Xu & Sato): loops excel at DAG evaluation, CoT at self-reducible problems
2025-06–10	HRM (27M parameters solves Sudoku/ARC), Mixture-of-Recursions, TRM, Ouro (2.6B, 7.7T tokens), Parallel Loop Transformer, parallel samplers (looped models ≈ diffusion LMs)
2025-11	Retrofitted Recurrence (McLeish); Think-at-Hard (loop only on hard tokens)
2026-01–04	LoopFormer (ICLR), Depth-Recurrent Attention Mixtures, RLTT (ICML), LoopRPT, Adaptive Loops and Memory, Hyperloop Transformers, iso-depth scaling law (φ=0.46), Loop-Think-Generalize (COLM)
2026-05	**Training-Free Looped Transformers** (UT Austin); MELT, LT2, LoopQ, STARS (ICML), Fully Looped Transformer, Attractor Models, Dual-Path
2026-06	**LoopCoder-v2** (7B PLT, 18T tokens, "only loop once"); Fixed-Point Reasoners; 1/N residual scaling theory; LOTUS closes the latent-vs-explicit CoT gap; Program-of-Layers (ICML)
2026-07	**Loop the Loopies!** (Loopie-20B-A2B, first compute-matched win); DeepLoop; Looped Latent Attention (32× KV compression); Nanbeige4.2-3B (28T-token looped model shipped as an agentic product); Hidden Decoding (WeChat's WeLM 617B takes the sequence-length route)
2026-08	MixerLoop (loop the mixer only), Gated Recurrent Transformers, Looped LMs improve compositional tool calling, Think Shallow Solve Deep (settling/drifting taxonomy)
2026-09	**SMELT**: Chinchilla-style scaling laws for compute-matched looped MoE, 6.8–18% training-FLOP savings; RecurTrace; GPT-6 Astra released, The Information reports recurrent depth

4. The five core papers at a glance

Full reports are in `reports/en/01–05`. A comparison table:

	Universal Transformers	Recurrent Depth (Huginn)	Training-Free Looped	LoopCoder-v2	Loop the Loopies
Date	2018-07	2025-02	2026-05	2026-06	2026-07
Group	UvA / DeepMind / Google	ELLIS / UMD / LLNL	UT Austin	Beihang / IQuest	IQuest
Granularity	model-loop	block (three-stage)	middle 4 layers, block or layer	model-loop (PLT)	layer-loop
Training	from scratch	from scratch, 800B tokens	**none**	from scratch, 18T tokens	from scratch, 3.5T tokens
Loop count	ACT adaptive	random $\bar{r}$=32 at train, free at test	K=3-stage RK	R=2 (train = test)	R=2
Scale	~65M	3.5B dense	0.6B–30B (existing models)	7B dense	20B-A2B / 6B-A0.6B MoE
Headline number	bAbI error 22.1→0.29	r 4→32: ARC-E 49.1→69.9	45 cells 87% non-negative, MMLU-Pro +2.64	SWE-bench Verified 43.0→64.4→27.6	overtakes Qwen3-30B-A3B after 600B tokens at matched compute
Core contribution	defines recurrent depth + halting	first modern-scale training; latent orbits	layer = Euler step; sub-step, don't repeat	mechanistic diagnosis of two-loop saturation; CoT + loop super-additive	Loopie Recipe; SPT
Most misread point	–	the "same 0.8T" baseline is really 0.18T	gains cap at ~2–3 points	non-monotonicity is PLT-specific	"compute-matched" = measured step time, not FLOPs

4.1 How the five fit together

The five papers form one argument:

1. UT proposes the mechanism; parameter–compute ratio and training stability remain open.
2. Huginn answers "can it be trained at modern scale" — yes, at the cost of inference compute and knowledge capacity.
3. Training-Free answers "can existing models get it for free" — partly — and supplies the ODE explanation and the layer-mode requirement for MoE.
4. LoopCoder-v2 answers "how many loops at deployment" — flattens latency with PLT, finds saturation at R=2, and diagnoses why.
5. Loopie answers "can it win at matched compute" — yes, via layer-loop + MoE + measured step-time matching, also at R=2.

5. The wider literature map

Categories below come from arXiv searches (looped/recurrent-depth/depth-recurrent transformer; latent reasoning □ loop/recurrence/recursive; named models), keeping work directly relevant to LM architecture. The full list is in the README and `data/papers.csv`.

5.1 Theory: expressivity, ICL, length generalization

- Giannou et al. 2023 (ICML): looped transformers as programmable computers.
- Yang et al. 2024 (ICLR): looped models learn learning algorithms with <10% of the parameters.
- Gatmiry et al. 2024 (ICML): can looped transformers implement multi-step GD; Chen et al. 2025 (AISTATS): bypassing the exponential dependency.
- Fan et al. 2025 (ICLR): adaptive-step looped transformers substantially improve length generalization.
- Saunshi et al. 2025 (ICLR): many reasoning problems need depth, not parameters; k layers looped L times ≈ kL layers; looped models simulate T steps of CoT.
- Xu & Sato 2025: formal separation — loops for deterministic parallel computation (DAG evaluation), CoT for approximate inference over self-reducible problems.
- Gong et al. 2025: loss-landscape view — looping induces a River-V-Valley bias.
- Zhang et al. 2026: the "budget law": weight-tied loops learn algorithms with speed v ≈ n_train/T_train; halting rule T* = □n/v̂□.
- Wu et al. 2026: looped transformers with LayerNorm provably learn the power method.

5.2 Scaled pretraining

Model	Size	Tokens	Loops	Notes
Huginn-0125	3.5B dense	0.8T	r̄=32, free at test	first modern scale; open
Ouro 1.4B / 2.6B	dense	7.7T	4 + entropy-regularized depth allocation	matches 12B SOTA; gains from knowledge manipulation, not storage; open
LoopCoder-v2	7B dense PLT	18T	2	coder; open
Loopie-20B-A2B / 6B-A0.6B	MoE	3.5T	2 (layer-loop)	first compute-matched win; open
Nanbeige4.2-3B	3B	28T	2 (stack reuse)	agentic product model, beats Qwen3.5-9B and Gemma4-12B
SMELT	MoE to 54B non-embedding	–	middle half ×2	scaling law: saves 6.8–18% training FLOPs

Common to scaled pretraining: **loop counts are small (2–4)** except Huginn; **the three newest are layer-loop or mid-stack**; **MoE backbones are the default**.

5.3 Efficiency: latency, KV memory, quantization

- **Latency**: Parallel Loop Transformer (cross-loop position offset, latency ≈ one forward); Efficient Parallel Samplers (Geiping 2025) treat looped models as diffusion LMs, a diffusion-forcing sampler gives 5× speedup with no tuning; Continuous Depth-wise Batching (Bae 2025).
- **KV memory**: Huginn's looped KV sharing (budget 4, no loss); Mixture-of-Recursions' KV-sharing variant; MELT (one shared KV per layer with gated updates, constant memory after fine-tuning Ouro); **Looped Latent Attention** (cross-loop

KV trajectories are low-rank; SVD codec near-lossless at 32×; Ouro-1.4B batch capacity on one H200 from 32 to 768); CHASE (skip-exit for looped SSMs).
- **Attention complexity**: LT2 (linear/sparse attention + loops; looping lets linear attention refine memory iteratively and sparse attention expand its receptive field; Ouro-hybrid-1.4B converted with ~1B tokens).
- **Quantization**: LoopQ (first systematic study; under W4A4, +68.8% average accuracy over the strongest PTQ baseline).
- **Deployment notes**: Halloran 2026 fixes five bugs in Nanbeige4.2-3B on Apple Silicon, notes that layer reuse doubles peak attention memory, and extends usable context 2.7× with chunked prefill.

5.4 Stability and training theory

- Huginn's two failed runs (hidden-state collapse, recurrence collapse) and sandwich normalization.
- Wang et al. 2026 (residual scaling): weight sharing correlates residual updates across iterations, requiring ε=1/N rather than 1/√L; factored ε=λ/(N√L); optimal LR depends only on the number of unique layers L, so it transfers from small to large N.
- DeepLoop: the Post-LN DeepNorm exponent must rise from 1/4 to 1/2; $\alpha = (2N)^{1/2}$, $\beta = (8N)^{-1/2}$.
- Fully Looped Transformer: distribute inter-loop signals to all layers + attention injection; stable to 12 loops.
- STARS (ICML 2026): Jacobian spectral-radius regularization + random loop sampling drives latent states to asymptotically stable fixed points, mitigating "peak then collapse".
- Fixed-Point Reasoners: pre-norm + residual scaling + fixed-point convergence as halting.
- Think Shallow, Solve Deep: an operator's finite-time regime (settling / marginal / drifting) predicts whether extra depth is safe; one terminal fixed-point objective moves regime and depth behavior together; Huginn is non-settling.
- Stabilizing Extrapolation via Learned Stochastic Stopping: randomizing loop count in training sharply reduces OOD variance.

5.5 Adaptive computation and halting

- Classics: ACT (2016), PonderNet (2021), CALM (2022).
- Mixture-of-Recursions (NeurIPS 2025): lightweight routers assign per-token recursion depth; attention and KV only for active tokens; a new Pareto frontier at 135M–1.7B.
- Think-at-Hard (ICML 2026): "latent overthinking" — most tokens are right after one pass and later iterations sometimes break them; loop only on the hard 7% of tokens, +3.8–4.4%.
- LoopFormer (ICLR 2026): shortcut-consistency training aligns trajectories of different lengths for budget-elastic inference.
- RecurTrace (Sep 2026): loop-memory attention (attend to one's own earlier loop states) + an oracle-supervised halting head; 56.9% on MathQA at 2.0 loops on average, while ACT/PonderNet collapse to one loop.
- Adaptive Depth in Looped Transformers (Popescu): diagnosing learned halting gates.
- Two-Scale Latent Dynamics: an early-exit rule on the second-order step-size difference beats Huginn's KL criterion.

5.6 Retrofit and training-free

- Relaxed Recursive Transformers (ICLR 2025): convert Gemma 2B and others into a shared block + per-layer LoRA, recovering most performance.
- Retrofitted Recurrence (McLeish 2025): a curriculum that gradually raises loop count; beats post-training the original on math at equal compute.
- Training-Free Looped Transformers (2026): zero training, RK sub-steps on the middle 4 layers.
- Program-of-Layers (ICML 2026): a small network emits a per-input program of which layers to skip or repeat, often more accurate with fewer layers.
- Retrofitting Recurrent Depth (Shapiro 2026): installation, extrapolation, transfer, and retention at two parameter budgets.

5.7 Post-training and reinforcement learning

- LoopCoder-v2: a Thinking mode (explicit CoT) on top of the R=2 loop, LiveCodeBench +26.9, super-additive.
- Loopie: SPT (SFT masking + PT scale, 2T tokens over 10 epochs without overfitting or forgetting) + GSPO/DAPO RL.
- RLTT (ICML 2026): standard GRPO credits only the final latent state, mismatching the loop's internal computation; distributing reward along the latent trajectory gives Ouro-2.6B +10.9% on math.
- LoopRPT: reframes next-token prediction as reasoning and assigns RL signal directly to latent steps via an EMA teacher, compressing effective reasoning into fewer iterations.
- LOTUS (Fan 2026): K latent blocks processed in parallel with a cross-entropy loss on each position's gold CoT token;

first latent-CoT method to match explicit CoT at 3B, cutting thought-phase latency 2.5–6.9×.
- LoopMTP: softly aligns loop-t hidden state with the token t steps ahead; stable to 15 loops.
- Dense Supervision Is Not Enough: the "readout blind spot" of looped LMs.

5.8 Interpretability and dynamics

- Huginn paper: orbits, sliders, fixed points; path independence.
- Lu et al. 2025: logit-lens probes of Huginn find limited evidence of interpretable latent CoT; probe results depend heavily on layer and decoding method.
- Two-Scale Latent Dynamics: small within-block refinements plus larger between-block drifts; loop steps shrink and become orthogonal over training.
- Jacobian Lens (Sep 2026): Ouro rebuilds a "global workspace" every loop and linear transport cannot cross loop boundaries; Huginn carries content across 16 recurrences but reads/writes act within a ~2-recurrence window.
- LoopCoder-v2: effective rank peaks at loop 2 then declines; update directions oscillate.
- Observable Patterns Are Not Explanations: a causal–geometric analysis warning against reading observable patterns as explanations.
- Step-resolved data attribution for looped transformers.

5.9 Tiny recursive reasoners

A parallel branch that does not do language modeling but solves hard structured puzzles with millions of parameters:

- HRM (Jun 2025): 27M parameters, 1000 samples, two recurrent modules at different frequencies; near-perfect Sudoku/Maze, beats larger models on ARC.
- TRM (Oct 2025): a single 2-layer 7M network; ARC-AGI-1 45%, ARC-AGI-2 8%, above most LLMs.
- Attractor Models (May 2026): fixed-point implicit differentiation, training memory independent of effective depth; 27M parameters 91.4% on Sudoku-Extreme; a 770M model beats a 1.3B transformer trained on 2× tokens; "equilibrium internalization" lets the solver be removed at inference.
- Denoising Recursion Models: multi-step denoising recursion; beats TRM on ARC-AGI.
- Critical analyses: Hierarchical Reasoning Models: Perspectives and Misconceptions; Are Your Reasoning Models Reasoning or Guessing.

5.10 Cross-modal and applications

- Vision: LoopViT (18M parameters, 65.8% on ARC-AGI-1, beating 73M ensembles), RD-ViT (semantic segmentation), ELT (visual generation), Déjà View (multi-view 3D reconstruction).
- Robotics: Recurrent-Depth VLA (tasks at 0% success with one iteration exceed 90% with four; 80× faster than token-reasoning VLAs).
- Speech: Depth-Conditioned Looped Transformers for ASR.
- Agents: Looped LMs Improve Compositional Tool Calling (multi-step tool use rises with recurrent depth); Nanbeige4.2-3B's agentic deployment.
- Multimodal LMs: Looping Back to Move Forward (recursive LMMs).

6. Ten trend insights

Insight 1: Recurrent depth passed the compute-matching gate — through a narrow door

Loopie (Jul 2026) and SMELT (Sep 2026) are the first wins against vanilla models at **matched training cost**: Loopie matches measured step time and overtakes a reproduced Qwen3-30B-A3B after 800B tokens; SMELT matches FLOPs, parameters, and KV simultaneously, fits scaling laws, saves 6.8–18% of training FLOPs, and the advantage does not vanish with scale (to 54B non-embedding parameters). Together they move looping from a parameter-efficiency trick to a compute-efficiency one.

The winning configuration is narrow: **MoE backbone, layer-loop or mid-stack looping, two loops.** Schwethelm et al.'s iso-depth scaling law explains why: one recurrence is worth $r^{0.46}$ unique parameters (0.38 under truncated backprop), so substituting loops for parameters loses under FLOP matching. Loopie's win does not trade parameters for loops; it converts the **activation memory** saved by looping into larger micro-batches and then into width — an infrastructure dividend invisible in theoretical FLOPs.

Insight 2: "How many loops" splits into two regimes

The reported optima side by side: Loopie 2, LoopCoder-v2 2, SMELT 2, Nanbeige4.2 2, Ouro 4, Huginn 32 (up to 64), RecurTrace 2.0 on average, Think-at-Hard 1 on 93% of tokens.

The dividing line is **where the compute constraint binds**. Under a fixed pretraining budget, each extra loop multiplies training FLOPs and the marginal return decays fastest after R=2 (Loopie Figure 6): the answer is 2. With variable inference compute, one model can loop more on hard inputs and less on easy ones (Huginn's ARC-C saturates at 8 loops zero-shot and 32 loops with 25–50 shots): the answer is "as needed, adaptively".

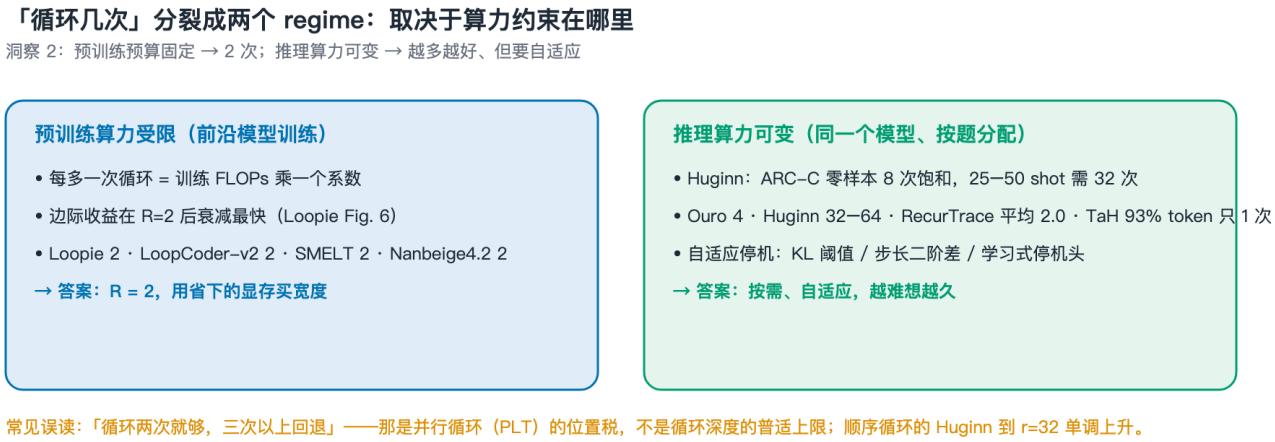


Figure 2 (labels in Chinese): left, fixed pretraining budget → two loops (Loopie, LoopCoder-v2, SMELT, Nanbeige); right, variable inference compute → adaptive and deep (Huginn 8 → 32 loops with more context; Ouro 4; RecurTrace 2.0 avg; Think-at-Hard loops only 7% of tokens).

LoopCoder-v2's "three or more regress" is often quoted as a universal ceiling. The paper's own explanation is that PLT's cross-loop position offset charges a fixed positional tax per loop, 30–45× the marginal gain after loop 2; sequential looping has no such term. It is the **price of parallelization**, not a property of looping.

Insight 3: Loop granularity is getting finer and converging on the middle

From UT's whole-model loop, to Huginn's three-stage design (ends unlooped), to Loopie/Training-Free/SMELT's per-layer or mid-stack loops, to MixerLoop looping only the sequence mixer while the FFN runs once (retaining 41.5% of the gain at 45.9% fewer looped FLOPs), granularity keeps shrinking. Three independent lines point to the same placement rule: Training-Free's depth-fraction rule (optimal window centered at fractional depth 0.43–0.71, mode 0.5), SMELT's "middle half", and Huginn's design rationale that early and late layers differ functionally from the homogeneous middle. Ends embed and read out and are specialized; the middle is homogeneous and reusable.

For MoE there is a hard reason for layer-loop: block-loop routes to different experts on each iteration (routing thrash). Training-Free (no training) and Loopie (from scratch) agree independently.

Insight 4: ODE flow or fixed-point iteration? Two philosophies are diverging

Training-Free's finding is that the mid-stack of a pretrained transformer "is not contractive in any useful sense": Anderson, heavy-ball, and Aitken accelerators all underperform simple RK sub-stepping. For a **trained** model the right loop refines the integral over [0,1] (damped Euler $x \leftarrow (1-1/K)x+(1/K)g(x)$) rather than advancing to t=K.

The other branch **trains** the loop to be a fixed-point iteration: STARS via Jacobian spectral-radius regularization, Fixed-Point Reasoners via convergence-as-halting, Attractor Models via implicit differentiation. The motivation is stability: Viakhirev et al. classify operators as settling / marginal / drifting; settling operators never degrade with depth, and a single terminal fixed-point objective turns drifting into settling. Huginn is non-settling.

The two philosophies fit two situations: follow the ODE flow when retrofitting an existing model; design stability into the objective when training from scratch. LoopCoder-v2's oscillating update directions ($\cos\theta<0$) are exactly non-settling behavior.

Insight 5: Training stability went from folklore to theorems

Huginn's two failed runs were lessons in 2025; in 2026 they are theorems: residual scaling needs 1/N not 1/√L (Wang et al.), the DeepNorm exponent must rise from 1/4 to 1/2 (DeepLoop), and the optimal LR depends only on the number of unique layers and transfers across loop counts. In practice, Fully Looped Transformer trains stably to 12 loops and LoopMTP to 15. With SmallInit and sandwich normalization, the "looped models are hard to train" obstacle standing since 2018 is essentially dismantled.

Insight 6: Inference cost is the last big bottleneck, now decomposed into three sub-problems

Looped inference costs three things: latency (serial loops), KV memory (one copy per loop), and quantization fragility. Each has 2025–2026 solutions: latency via PLT's parallelization or Geiping's diffusion-style parallel sampler (5×); KV via sharing (Huginn, MoR, MELT) or compression (Looped Latent Attention finds cross-loop KV low-rank, 32× near-lossless); quantization via LoopQ. These are independent and stackable.

One accounting caveat: Loopie's "compute-matched" is matched training step time; at inference it executes 54 block passes per token versus the reference's 48 with a larger KV cache, and the paper reports no inference latency. **A win at matched training compute is not automatically a win at matched inference cost.**

Insight 7: Latent looping and explicit CoT are complements, not substitutes

Three pieces of evidence: LoopCoder-v2's Thinking mode on top of R=2 gives LiveCodeBench +26.9, beyond the sum of the parts; LOTUS supervises latent positions with gold CoT tokens and is the first to match explicit CoT at 3B while cutting thought latency 2.5–6.9×; Xu & Sato's formal separation shows each excels on a different problem class.

The companion finding is that **standard RL is mismatched to looped models**: GRPO credits only the final latent state (RLTT identifies this as why RL on looped models kept failing), and distributing reward along the latent trajectory gives Ouro-2.6B +10.9% on math; LoopRPT assigns RL signal to latent steps already in pretraining. Post-training a looped model needs redesigned objectives; recycling CoT-era RL recipes wastes the loop.

Insight 8: The capability profile is now clear — loops manipulate knowledge, they do not store it

Ouro's controlled experiments attribute its advantage to knowledge manipulation, not capacity; Huginn is weak closed-book and strong open-book (OBQA +11.0); Frey et al. find looping helps math while memory banks help commonsense, and only the combination beats an iso-FLOP baseline; Dual-Path runs a "deep" looped sublayer and a "wide" FFN in parallel and learns gates that send punctuation, symbols, and arithmetic tokens deep while function words and lexical content go wide.

This defines the loop's best partners: **retrieval, long context, MoE expert banks** — anything that externalizes storage. Loopie and SMELT choosing MoE backbones, and Adaptive Loops and Memory adding gated memory banks, follow this logic.

Insight 9: Applications are expanding from math puzzles to agents and embodiment

Evidence in 2025 concentrated on Sudoku, mazes, ARC, GSM8K; 2026 adds three arenas: (1) **software-engineering agents** — LoopCoder-v2's SWE-bench Verified +21.4 and large Terminal-Bench gains; (2) **tool calling** — Popescu et al. find multi-step, dependency-aware tool use rises with recurrent depth, and Nanbeige4.2-3B beats Qwen3.5-9B on agentic benchmarks at 3B; (3) **robotics** — RD-VLA's looped action head takes some tasks from 0% to >90% and runs 80× faster than token-reasoning VLAs. The common thread: these tasks require maintaining intermediate state and dependencies within one forward pass, which is what loops are for.

Insight 10: Productization has started; the toolchain has not caught up

Five open looped-model families exist — Huginn, Ouro, LoopCoder-v2, Loopie, Nanbeige4.2-3B — and Nanbeige4.2-3B was trained on 28T tokens and shipped as a product. Yet Halloran's replication report finds five out-of-the-box bugs running it in Hugging Face transformers (including a silently zeroed RoPE buffer and calls to removed cache APIs), and even after fixes, layer reuse doubles attention memory. Inference frameworks, quantization tools, and KV management have no standard interface for looped models. Loopie shipping megatron-loopie and vllm-loopie branches alongside the paper is a step in that direction.

7. Inferences about GPT-6 Astra (explicitly labeled as inference)

The following rest on the public literature surveyed here and on **no** first-hand information about Astra.

1. If Astra uses recurrent depth, the recipe in the public literature closest to "frontier scale + compute-matched" is the Loopie/SMELT line: MoE backbone, mid-stack or per-layer looping, small loop count. The Huginn line of looping a whole model 32 times has no compute-matched evidence at frontier pretraining budgets.
2. The common speculation that "OpenAI found a way around two-loop saturation" starts from a false premise — two-loop saturation is PLT-specific. Sequential looped models (Huginn) never had that bottleneck; the real one is inference cost, which public methods such as 32× KV compression and 5× diffusion-style parallel sampling have already cut by an order of magnitude.
3. The "security concerns" mentioned in The Information's report match a real issue in the literature: latent reasoning produces no readable intermediate tokens, and interpretability work (Lu 2025, Jacobian Lens 2026) shows that probes of looped models' internal states depend on layer and decoding method — supervising the reasoning process is harder than supervising CoT.
4. On timing, Loopie (July), Looped Latent Attention (July), SMELT (1 September), and Astra (early September) cluster tightly; the more reasonable reading is that several groups pushed this direction to usability in the first half of 2026 simultaneously, not that one paper caused one product.

8. Open problems

1. **Compute-matched multi-loop.** Every compute-matched win is at R=2. Whether R≥4 can win under matched pretraining compute has no public evidence.
2. **The full inference bill.** Training step time, inference latency, and peak KV memory need to be reported together; no paper yet does all three.
3. **Scaling laws for looped models.** SMELT fits to 54B non-embedding parameters, Schwethelm to ~1B. Whether the functional form (Chinchilla plus an r^{φ} term) holds at 100B+ is unknown.
4. **Adaptive halting at pretraining scale.** MoR, TaH, and RecurTrace results are all ≤8B; Ouro's entropy-regularized depth allocation is the only one validated at 7.7T tokens.
5. **Supervisability of latent reasoning.** LOTUS shows gold-CoT supervision of latent positions works, but requires CoT data; making latent trajectories readable and auditable without it is an open safety question.
6. **The mechanism of layer-loop.** Of Loopie's three reasons (scaling, infrastructure, local sharing) only scaling has experiments; SMELT's "the second visit reduces the attention sink" is the only mechanistic account so far.
7. **Looping versus sequence-length scaling.** Hidden Decoding (WeLM 617B) adds compute along the sequence dimension and claims better pipeline-parallel compatibility; Loopie's layer-loop was designed precisely for pipeline compatibility. No head-to-head at 100B+ exists.

9. Conclusion

In eight years recurrent depth went from an encoder–decoder trick for bAbI overfitting to a candidate component of frontier architectures. Three turns in the first half of 2026 — training-free looping giving existing models 2 free points, PLT keeping latency flat as loops grow, and Loopie/SMELT winning at matched compute for the first time — pushed it past the engineering-feasibility threshold. What remains is a full accounting of inference cost, compute-matched evidence for more than two loops, and supervisability of latent reasoning. For researchers the most valuable gaps are compute-matched scaling laws at R≥4 and loop-specific RL objectives; for engineering teams the practical conclusion is: loop the middle, twice, per layer, on an MoE backbone — and measure latency and KV before shipping.

Appendix A: Circulation errors corrected in this report

Common claim	What the paper says
Huginn crushes its baseline "at the same 0.8T tokens"	The non-recurrent baseline trained for 0.18T; at equal 0.18T the looped model gets ARC-E 53.62 vs 46.42
"Two loops suffice, three regress" is a law of looped transformers	It is specific to PLT (cross-loop position offset); Huginn improves monotonically to r=32
Loopie and Qwen3-30B-A3B use "the same compute"	The same measured optimizer-step time; Loopie's nominal FLOPs are 42% higher and inference cost is unreported

Common claim	What the paper says
Training-Free makes open models “stronger”	Gains of about 1–2.6 points, concentrated on knowledge MC; small distilled models can lose points
Huginn “performs like a 50B model at 3.5B”	FLOP-equivalent, not benchmark-equivalent; on benchmarks it is near OLMo-7B and behind OLMo-2

Appendix B: Method and sources

- Five core papers: arXiv PDFs (`papers/original/`) and arXiv HTML full texts (`papers/src/`), read section by section.
- Chinese translations: SuperTranslate layout-preserving translation with object-level QA (`papers/zh/`; QA record in `QA_SUMMARY.md`).
- Related literature: arXiv API searches on 2026-09-06 combining looped/recurrent-depth/depth-recurrent transformer, latent reasoning □ (loop|recurrence|recursive), and named models; 60 deduplicated abstracts kept in `data/related_abstracts.json`.
- Writing rules: lead with the claim, state necessary limits once, make every number checkable; cite no primary source we could not access.

Appendix C: Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters applied repeatedly along depth; effective depth = stored layers × loop count
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next (Figure 1)
Input injection	Re-feed the input embedding at every loop; otherwise the iteration is unstable
Truncated backprop	Gradients through only the last k loops so memory is independent of loop count
Adaptive halting (ACT)	Each token decides how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; one extra copy per loop, the main memory bottleneck
MoE	Mixture of experts; “20B-A2B” = 20B total, 2B active parameters
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs measured training step time; looping’s activation-memory savings only appear in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as tokens
Fixed point	A state another loop leaves unchanged; one line of work trains loops to converge to fixed points
PLT / CLP	Parallel Loop Transformer / cross-loop position offset
φ in the iso-depth scaling law	How many unique parameters one recurrence is worth: 1 = full equivalence, 0 = no capacity gain; measured 0.46